

Volume 6 – Metadata Lakehouse Agent Implementations

Version: 1.0

Purpose

This document specializes the Agent Framework for the Metadata Lakehouse Platform.

Target Platform:

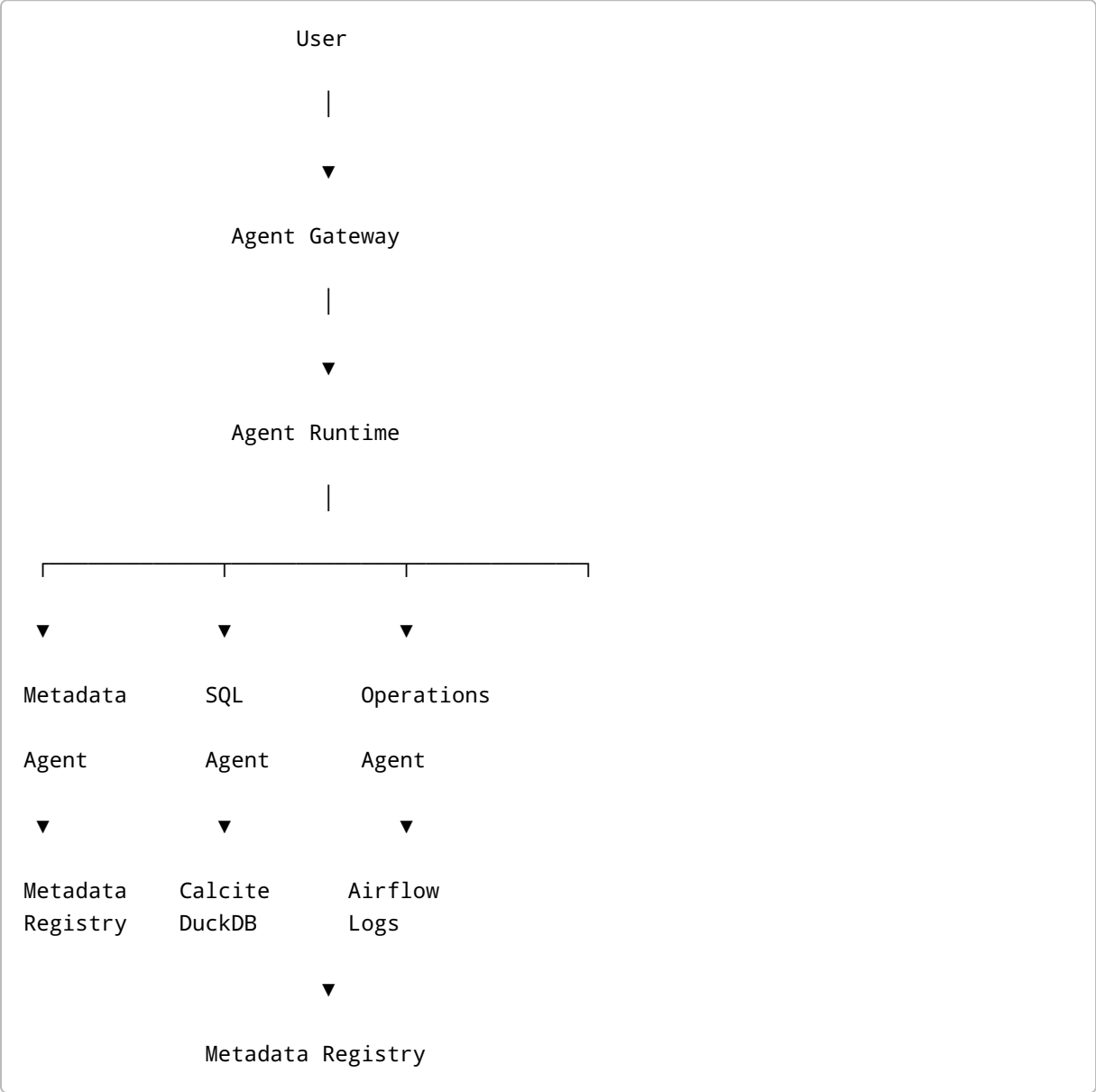
```
graph TD; A[Raw Metadata] --> B[Canonical Metadata Generator]; B --> C[Metadata Registry (DuckDB)]; C --> D[Apache Calcite]; D --> E[DuckDB Execution Engine]; E --> F[Parquet / CSV / Iceberg Data];
```

Raw Metadata
↓
Canonical Metadata Generator
↓
Metadata Registry (DuckDB)
↓
Apache Calcite
↓
DuckDB Execution Engine
↓
Parquet / CSV / Iceberg Data

The goal is to provide AI-assisted capabilities for:

- Metadata discovery
 - SQL generation
 - Rule authoring
 - Data quality investigation
 - ETL operations
 - Documentation
 - Research
-

Platform Architecture



Metadata Registry Model

The metadata registry is the foundation.

Every agent ultimately relies on it.

Registry Categories

Raw Metadata

Represents source structures.

Examples:

CSV

ASN.1

XML

JSON

TAP

Canonical Metadata

Represents business entities.

Examples:

Subscriber Usage

Recharge Event

Balance Adjustment

SMS Usage

Physical Metadata

Represents storage.

Examples:

Parquet

Iceberg

DuckDB Tables

Agent Catalog

The platform starts with seven core agents.

Agent 1

Metadata Discovery Agent

Business Capability

Find datasets.

User Examples

Find subscriber usage datasets.

Which table contains IMSI?

Show recharge-related entities.

Find all event types containing location information.

Allowed Tools

SearchDatasetTool

GetDatasetTool

GetSchemaTool

GetLineageTool

Preferred Model

Qwen 4B

Output

```
{
  "datasets": [
    {
      "name": "subscriber_usage",
      "confidence": 0.95
    }
  ]
}
```

Execution Flow

```
graph TD
    A[User Query] --> B[Metadata Agent]
    B --> C[SearchDatasetTool]
    C --> D[Registry Lookup]
    D --> E[Response]
```

Agent 2

SQL Authoring Agent

Business Capability

Generate valid SQL.

User Examples

Show top recharge amounts.

Find inactive subscribers.

Calculate monthly usage.

Allowed Tools

Metadata Agent APIs

GetSchemaTool

CalciteValidationTool

SqlExplainTool

Preferred Model

Qwen 4B

Fallback

Qwen 14B

Output

```
{
  "sql": "...",
  "validated": true,
  "confidence": 0.92
}
```

Important Rule

This agent never executes SQL.

Execution belongs elsewhere.

Execution Flow

```
User Request
  ↓
Metadata Retrieval
  ↓
SQL Generation
  ↓
Calcite Validation
  ↓
Response
```

Agent 3

Query Execution Agent

Business Capability

Execute SQL.

User Examples

Run query.

Preview results.

Get row counts.

Allowed Tools

DuckDBExecuteTool

QueryLimitTool

ResultFormatterTool

Preferred Model

Small Local Model

Output

```
{  
  "rows": [],
```

```
"executionTimeMs": 320
}
```

Security

Should enforce:

```
Maximum Rows
Execution Timeout
Allowed Schemas
```

Agent 4

Rule Authoring Agent

Business Capability

Create business rules.

User Examples

```
Detect duplicate IMSI.
Find recharge spikes.
Identify missing usage records.
```

Allowed Tools

```
Metadata Search
Rule Template Tool
```

Calcite Validation

Output

```
{  
  "ruleName": "...",  
  "ruleSql": "...",  
  "severity": "HIGH"  
}
```

Integration

Generates artifacts for:

Rule Repository

Execution Scheduler

Case Management

Agent 5

Data Quality Agent

Business Capability

Investigate data quality issues.

User Examples

Why is recharge count lower today?

Which files are missing?

Find null IMSI records.

Allowed Tools

File Metadata Tool

Dataset Statistics Tool

Query Execution Tool

Quality Rule Tool

Output

```
{  
  "rootCause": "...",  
  "confidence": 0.84  
}
```

Example Flow

```
Issue  
  ↓  
Dataset Statistics  
  ↓  
File Analysis  
  ↓  
Quality Checks  
  ↓  
Recommendation
```

Agent 6

ETL Operations Agent

Business Capability

Manage platform operations.

User Examples

Why did my job fail?

Show today's failed workflows.

Restart staging pipeline.

Allowed Tools

GetJobStatusTool

GetJobLogsTool

RestartJobTool

ExecutionHistoryTool

Security

Separate:

Read Operations

Write Operations

Administrative Operations

Example Output

```
{  
  "jobId": "12345",  
  "status": "FAILED",  
  "reason": "Missing source file"  
}
```

Agent 7

Documentation Agent

Business Capability

Explain platform behavior.

User Examples

Explain subscriber usage model.

How does canonical mapping work?

What is event lineage?

Allowed Tools

Document Search

Metadata Search

Architecture Repository

Output

```
{  
  "answer": "...",  
  "references": []  
}
```

Supporting Agent

Research Agent

Purpose

External information retrieval.

User Examples

```
Research Iceberg partitioning.  
  
Compare DuckDB vs Trino.  
  
Latest Calcite improvements.
```

Allowed Tools

```
Web Search Tool  
  
Document Reader Tool  
  
Summarization Tool
```

Preferred Model

Qwen 14B

Metadata Registry Tools

SearchDatasetTool

Purpose:

Find datasets

Input:

```
{  
  "searchText": "subscriber"  
}
```

Output:

```
{  
  "datasets": []  
}
```

GetDatasetTool

Purpose:

Retrieve dataset details

GetSchemaTool

Purpose:

Retrieve columns

GetLineageTool

Purpose:

Retrieve lineage graph

SQL Tools

GenerateSqlTool

Purpose:

Create SQL

ValidateSqlTool

Purpose:

Calcite Validation

ExplainSqlTool

Purpose:

Generate SQL explanation

DuckDB Tools

ExecuteQueryTool

Purpose:

Run SQL

DescribeTableTool

Purpose:

Retrieve table metadata

DatasetStatisticsTool

Purpose:

Generate profiling statistics

Operations Tools

GetJobStatusTool

Purpose:

Workflow status

RestartJobTool

Purpose:

Restart execution

Approval required.

GetJobLogsTool

Purpose:

Retrieve logs

Metadata Registry Schema

Recommended tables.

metadata_dataset

```
CREATE TABLE metadata_dataset (  
    dataset_id BIGINT,  
    dataset_name VARCHAR,  
    business_name VARCHAR,  
    description VARCHAR  
);
```

metadata_column

```
CREATE TABLE metadata_column (  

```

```
column_id BIGINT,  
  
dataset_id BIGINT,  
  
column_name VARCHAR,  
  
data_type VARCHAR  
);
```

metadata_lineage

```
CREATE TABLE metadata_lineage (  
  
    source_dataset_id BIGINT,  
  
    target_dataset_id BIGINT  
);
```

metadata_rule

```
CREATE TABLE metadata_rule (  
  
    rule_id BIGINT,  
  
    rule_name VARCHAR,  
  
    sql_definition VARCHAR  
);
```

Agent Capability Registry

Recommended registry.

```
capabilities:  
  
  dataset.search:  
    agent: metadata-agent
```

```
dataset.schema:
  agent: metadata-agent

sql.generate:
  agent: sql-agent

sql.execute:
  agent: query-agent

rule.generate:
  agent: rule-agent

quality.investigate:
  agent: data-quality-agent

job.status:
  agent: operations-agent

documentation.search:
  agent: documentation-agent

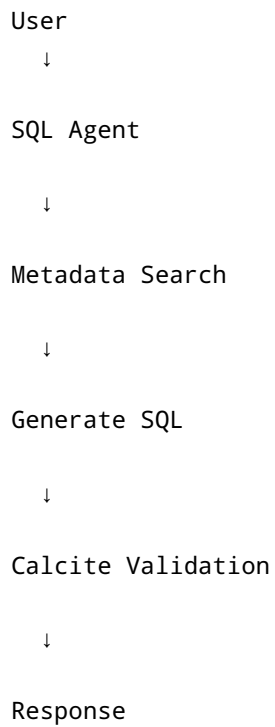
research.external:
  agent: research-agent
```

Typical End-to-End Use Cases

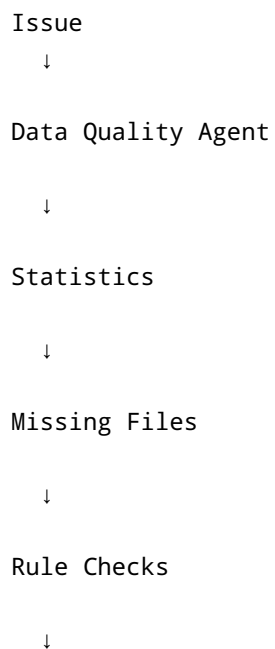
Dataset Discovery

```
User
↓
Metadata Agent
↓
SearchDatasetTool
↓
Metadata Registry
```

SQL Generation



Data Quality Investigation



Root Cause

ETL Failure Analysis

Failed Job

↓

Operations Agent

↓

Execution History

↓

Logs

↓

Root Cause Summary

Recommended Phase 1 Implementation

Implement first:

Metadata Agent

SQL Agent

Query Agent

Tools:

SearchDatasetTool

GetSchemaTool

ValidateSqlTool

ExecuteQueryTool

This immediately delivers:

Natural Language

↓

Metadata Discovery

↓

SQL Generation

↓

SQL Execution

which provides the highest business value with the lowest implementation effort.

Recommended Phase 2

Add:

Rule Authoring Agent

Data Quality Agent

Documentation Agent

Recommended Phase 3

Add:

Operations Agent

Research Agent

Final Architecture Principle

The Metadata Registry is the brain.

Agents are specialists.

Tools perform work.

Models provide reasoning.

DuckDB executes data operations.

Calcite validates intent.

The platform should remain metadata-driven so that new datasets, event types, and business entities can be added through metadata rather than code changes.